

Unit 12. Ensembles

1 Bagging

ISLR §8.2.1.

Unit 11 ended with a measurement: two trees grown on disjoint halves of **Boston** agree on the root variable under a fifth of the time and their predictions differ by about 45% of the response's own standard deviation. That is variance, and Unit 1's decomposition says variance is the part of the error that averaging can remove.

1.1 Averaging

Theorem 12.1 (Variance of an average of equicorrelated predictors). Let $Z_1, \dots, Z_B$ have common variance σ^2 and common pairwise correlation ρ . Then

$$\text{var}\left(\frac{1}{B} \sum_{b=1}^B Z_b\right) = \rho \sigma^2 + \frac{1-\rho}{B} \sigma^2, \quad (1)$$

and necessarily $\rho \geq -1/(B-1)$. For $\rho \geq 0$ the right side decreases in B to the limit $\rho \sigma^2$.

Proof. $\text{var}(\sum_b Z_b) = \sum_b \text{var}(Z_b) + \sum_{a \neq b} \text{cov}(Z_a, Z_b) = B\sigma^2 + B(B-1)\rho\sigma^2$, since each of the $B(B-1)$ ordered pairs has covariance $\rho\sigma^2$. Dividing by B^2 ,

$$\text{var}(\bar{Z}) = \frac{\sigma^2}{B} + \frac{B-1}{B}\rho\sigma^2 = \rho\sigma^2 + \frac{\sigma^2(1-\rho)}{B},$$

which is (1). A variance is non-negative, so $B\sigma^2 + B(B-1)\rho\sigma^2 \geq 0$ forces $\rho \geq -1/(B-1)$. For $\rho \geq 0$ only the second term of (1) depends on B and it decreases. $\square$

Remark. Everything in this unit is a comment on (1). The second term is what averaging removes and it vanishes at rate $1/B$, so the number of members is not a tuning parameter: more is never worse. The first term is what averaging cannot touch. Bagging accepts $\rho\sigma^2$ and drives B up; a random forest attacks ρ itself (§2); boosting abandons the averaging picture entirely (§3).

Corollary 12.2 (The same identity for sample moments). Let $\mathbf{Q}$ be an $M \times B$ matrix, $\hat{s}_b^2$ the sample variance of its b th column and $\hat{c}$ the average of the $B(B-1)/2$ distinct sample covariances

between columns. Then the sample variance of the row means is exactly

$$\frac{1}{B} \overline{\hat{s}^2} + \frac{B-1}{B} \hat{c}, \quad \overline{\hat{s}^2} = \frac{1}{B} \sum_b \hat{s}_b^2. \quad (2)$$

Proof. The row means are $\mathbf{Q}\mathbf{1}/B$, whose sample variance is $B^{-2}\mathbf{1}'\hat{\Sigma}\mathbf{1}$ with $\hat{\Sigma}$ the sample covariance matrix of the columns. Expanding, $\mathbf{1}'\hat{\Sigma}\mathbf{1} = \sum_b \hat{s}_b^2 + 2 \sum_{a < b} \hat{c}_{ab} = B\overline{\hat{s}^2} + B(B-1)\hat{c}$. $\square$

Remark. Corollary 12.2 is an algebraic identity, not an approximation, which is why §4.1 can check it to machine precision. Theorem 12.1 is the same statement about population moments and needs the equicorrelation hypothesis; for an ensemble that hypothesis holds by symmetry, the members being built by identical independent randomisations of one procedure.

1.2 The bootstrap ensemble

Definition 12.3 (Bagging). Let $\mathcal{D}^{*1}, \dots, \mathcal{D}^{*B}$ be independent bootstrap samples of the training data (Unit 1) and let $\hat{f}^{*b}$ be the predictor fitted to $\mathcal{D}^{*b}$. The **bagged** predictor is

$$\hat{f}_{\text{bag}}(\mathbf{x}) = \frac{1}{B} \sum_{b=1}^B \hat{f}^{*b}(\mathbf{x}) \quad (3)$$

for regression, and for classification the class receiving the most votes among $\hat{f}^{*1}(\mathbf{x}), \dots, \hat{f}^{*B}(\mathbf{x})$. The trees are grown large and not pruned.

Proposition 12.4 (Bagging moves variance, not bias). The $\hat{f}^{*b}(\mathbf{x})$ are identically distributed, so $\mathbb{E}[\hat{f}_{\text{bag}}(\mathbf{x})] = \mathbb{E}[\hat{f}^{*1}(\mathbf{x})]$: the ensemble has exactly the bias of one of its members. Its variance is (1) and is strictly smaller than a member's whenever $\rho < 1$.

Proof. Expectation is linear and the terms are identically distributed. For the variance, the difference between σ^2 and (1) is $(1 - \rho)\sigma^2(1 - 1/B)$, positive when $\rho < 1$ and $B > 1$. $\square$

Remark. This is why the trees are left unpruned. Pruning trades variance for bias (Unit 11), and the averaging in (3) already removes variance at no cost in bias, so the member to feed it is the one with the least bias available. The two devices would otherwise be paying twice for the same thing.

Remark. The bias is that of a tree fitted to a **bootstrap** sample, not to the original data. Proposition 12.4 says the averaging introduces no bias; it does not say the bagged predictor has the bias of a tree fitted to $\mathcal{D}$. Resampling is what makes the members differ, and paying a little bias for that is the trade bagging makes.

Proposition 12.5 (Majority vote under independence). Fix $\mathbf{x}$ and suppose the B classifiers are independent and each correct with probability $\pi > \frac{1}{2}$. Then the majority vote is correct with probability at least

$$1 - \frac{\pi(1 - \pi)}{B(\pi - \frac{1}{2})^2} \longrightarrow 1. \quad (4)$$

Proof. The number N of correct classifiers is binomial with mean $B\pi$ and variance $B\pi(1 - \pi)$. The vote is wrong only if $N \leq B/2$, that is $N - B\pi \leq -B(\pi - \frac{1}{2})$, and Chebyshev's inequality (prerequisites) bounds that probability by $B\pi(1 - \pi)/\{B(\pi - \frac{1}{2})\}^2$. $\square$

Remark. The hypothesis is independence, and ensemble members built from one data set are not independent: that is what $\rho > 0$ means. Proposition 12.5 is therefore an account of why voting could help, not a guarantee that it does, and the size of ρ decides how much of the promised gain survives. The same caution applies to (1), where $\rho\sigma^2$ is the part that no amount of voting removes.

1.3 Out-of-bag estimation

Definition 12.6 (Out-of-bag prediction). Let $\mathcal{O}_i = \{b : (\mathbf{x}_i, y_i) \notin \mathcal{D}^{*b}\}$ and $B_i = |\mathcal{O}_i|$. The **out-of-bag** prediction for observation i is $\hat{f}_{\text{oob}}(\mathbf{x}_i) = B_i^{-1} \sum_{b \in \mathcal{O}_i} \hat{f}^{*b}(\mathbf{x}_i)$, and the out-of-bag error is $n^{-1} \sum_i \{y_i - \hat{f}_{\text{oob}}(\mathbf{x}_i)\}^2$.

Theorem 12.7 (What out-of-bag estimates). (i) $\mathbb{E}[B_i] = B(1 - 1/n)^n$, which increases to $B/e \approx 0.368B$.

- (ii) Every tree averaged in $\hat{f}_{\text{oob}}(\mathbf{x}_i)$ was fitted without $(\mathbf{x}_i, y_i)$, so the prediction is out of sample.
- (iii) Conditionally on $B_i = B'$, the out-of-bag prediction has the same bias as the full ensemble and, under the hypotheses of Theorem 12.1, variance larger by

$$(1 - \rho)\sigma^2\left(\frac{1}{B'} - \frac{1}{B}\right), \quad (5)$$

so its mean squared error exceeds the full ensemble's by that amount, of order $(e - 1)(1 - \rho)\sigma^2/B$.

Proof. (i) By the out-of-bag fraction proposition of Unit 1 each bootstrap sample misses $(\mathbf{x}_i, y_i)$ with probability $(1 - 1/n)^n$, and B_i counts B independent such events.

(ii) is Definition 12.6.

(iii) The members are identically distributed, so an average of any B' of them has the common mean and, by Theorem 12.1 with B' in place of B , variance $\rho\sigma^2 + (1 - \rho)\sigma^2/B'$. Subtracting the same expression with B gives (5). Two predictors with the same expectation differ in mean squared error by exactly their difference in variance. Substituting $B' = B/e$ gives $(1 - \rho)\sigma^2(e - 1)/B$. $\square$

Remark. So the out-of-bag error is honest about being out of sample and is biased **upward** as an estimate of the full ensemble's error, by an amount that vanishes like $1/B$. At $B = 500$ that correction is negligible beside the variability of the estimate itself, which is what §4.2 measures: on one split of **Boston** the out-of-bag and test errors differ by a factor of 1.8, and averaged over twenty-five splits they differ by 12%.

1.4 Variable importance

Definition 12.8 (Mean decrease in impurity). For a tree T let $\Delta(t) = R(t) - R(t_L) - R(t_R)$ be the cost reduction at internal node t (Unit 11), non-negative by the refinement property. The **impurity importance** of predictor j in T is $\sum\{\Delta(t) : t \text{ splits on } j\}$, and in an ensemble it is the average of that over the members.

Proposition 12.9 (The importances of a tree add up to what it explains).

$$\sum_{t \text{ internal}} \Delta(t) = R(r) - R(T), \quad (6)$$

r being the root. So the impurity importances of all predictors sum to the total cost the tree

removes, and dividing by $R(r) - R(T)$ turns them into shares.

Proof. Write I for the internal nodes and L for the leaves. Every node except the root is a child of exactly one internal node, so $\sum_{t \in I} \{R(t_L) + R(t_R)\} = \sum_{t \neq r} R(t) = \sum_{t \in I} R(t) - R(r) + \sum_{t \in L} R(t)$. Subtracting this from $\sum_{t \in I} R(t)$ leaves $R(r) - \sum_{t \in L} R(t) = R(r) - R(T)$. $\square$

Definition 12.10 (Permutation importance). Let $\mathcal{E}$ be a set of observations not used in fitting and L the loss. The **permutation importance** of predictor j is

$$\frac{1}{|\mathcal{E}|} \sum_{i \in \mathcal{E}} L(y_i, \hat{f}(\mathbf{x}_i^{\pi(j)})) - \frac{1}{|\mathcal{E}|} \sum_{i \in \mathcal{E}} L(y_i, \hat{f}(\mathbf{x}_i)), \quad (7)$$

where $\mathbf{x}^{\pi(j)}$ is $\mathbf{x}$ with the j th coordinate replaced by that of a randomly chosen member of $\mathcal{E}$. In a bagged ensemble $\mathcal{E}$ is taken to be the out-of-bag set of each tree.

Proposition 12.11 (A useless predictor has zero permutation importance). If X_j is independent of (Y, X_{-j}) and the observations are independent and identically distributed, then permuting the

j th column leaves the joint law of $\mathcal{E}$ unchanged, so the population version of (7) is 0.

Proof. Under independence the joint law of $\mathcal{E}$ is the product of the law of (Y, X_{-j}) and the law of X_j , which is exchangeable in the X_j coordinate across observations. Permuting that coordinate therefore leaves the joint law unchanged, and the two averages in (7) have the same expectation. $\square$

Remark. No such statement holds for Definition 12.8. A predictor independent of the response still offers cutpoints, and the best of many cutpoints reduces the training cost by a positive amount; a predictor offering more distinct cutpoints reduces it by more. So impurity importance is positive for pure noise and increases with the number of values the predictor takes, which §5.2 exhibits with four noise variables whose impurity importances differ elevenfold.

2 Random Forests

ISLR §8.2.2.

2.1 Decorrelation

Theorem 12.1 leaves $\rho\sigma^2$ standing however large B is. Bagged trees are correlated for a plain reason: each is grown greedily from a resample of the same data, so if one predictor dominates the first split it dominates it in nearly every member, and the trees agree near the top.

Definition 12.12 (Random forest). A **random forest** is bagging with one change: at every split, a uniformly random subset of m of the p predictors is drawn, independently of everything else, and the split may use only those m . Taking $m = p$ recovers bagging. The default is $m = \lfloor p/3 \rfloor$ for regression and $m = \lfloor \sqrt{p} \rfloor$ for classification.

Proposition 12.13 (Availability). At any given split a fixed predictor is among the m candidates with probability m/p . Along a path of d consecutive splits it is available at none of them with probability $(1 - m/p)^d$, and at least once with probability $1 - (1 - m/p)^d$.

Proof. A uniformly random m -subset of a p -set contains a fixed element with probability $\binom{p-1}{m-1}/\binom{p}{m} = m/p$. The draws at distinct splits are independent. $\square$

Remark. With $p = 13$ and $m = 4$ a given predictor is available at 30.8% of splits, and the chance that the dominant one is shut out of all three splits nearest the root is $(9/13)^3 = 0.332$. A third of the trees are therefore forced to start somewhere else, which is the mechanism by which ρ falls.

Remark (The trade-off, read off (1)). Both quantities in Theorem 12.1 depend on m . Lowering m lowers ρ , which lowers the floor $\rho\sigma^2$; it also removes candidates from every split, which can only raise the training cost of each tree and generally raises σ^2 . The ensemble's limiting variance is the product $\rho(m)\sigma^2(m)$ of a term that increases in m and one that decreases, and nothing here says which effect wins. Nor is variance the whole error: restricting the candidates also changes the bias. §4.3 measures all of it and finds the variance smallest at $m = 1$, the squared bias smallest at $m = p$, and their sum smallest in between. The choice of m is made by the out-of-bag error.

2.2 What decorrelation costs

Remark. Restricting the candidate set is not free, and the case where it hurts is the one Unit 11 used to show greedy growing fail. For the exclusive-or truth every single split has zero population gain, and the structure appears only when a split on x_1 is followed by one on x_2 ; a forest that hides x_2 from most splits makes that pairing rarer. The measurement in §4.5 is that on a box truth in

two of six coordinates, bagging reaches a squared error of 0.36 against 1.27 for a forest with $m = 2$, while on a linear truth the two are indistinguishable. Decorrelation helps when the members are similar for a bad reason and hurts when they are similar because the problem requires it.

Remark. Two properties of a single tree survive the averaging and one does not. Monotone invariance of the predictors and the direct handling of categorical variables (Unit 11) hold for every member, hence for the ensemble. Interpretability does not: the object is now several hundred trees and the only summaries left are the importance measures of §1.4, which is why those measures carry the weight they do and why their biases matter.

3 Boosting

ISLR §8.2.3.

3.1 Fitting the residuals

Definition 12.14 (Boosting for regression). Fix a shrinkage parameter $\lambda \in (0, 1]$, a number of rounds B , and a class $\mathcal{G}$ of base learners, here trees with d splits. Set $\hat{f}^{(0)} \equiv 0$ and $r_i^{(0)} = y_i$. For $b = 1, \dots, B$, fit $\hat{g}^{(b)} \in \mathcal{G}$ to the data $(\mathbf{x}_i, r_i^{(b-1)})$ and put

$$\hat{f}^{(b)} = \hat{f}^{(b-1)} + \lambda \hat{g}^{(b)}, \quad r_i^{(b)} = r_i^{(b-1)} - \lambda \hat{g}^{(b)}(\mathbf{x}_i). \quad (8)$$

The output is $\hat{f}^{(B)} = \lambda \sum_{b \leq B} \hat{g}^{(b)}$. No resampling is involved and the members are built in sequence.

Proposition 12.15 (Why the residuals). With squared error loss, the stagewise problem $\min_{g \in \mathcal{G}} \sum_i \{y_i - \hat{f}^{(b-1)}(\mathbf{x}_i) - g(\mathbf{x}_i)\}^2$ is exactly the least squares fit of g to the current residuals $r_i^{(b-1)}$, and $r_i^{(b)} = y_i - \hat{f}^{(b)}(\mathbf{x}_i)$ holds at every stage.

Proof. The objective is $\sum_i \{r_i^{(b-1)} - g(\mathbf{x}_i)\}^2$ by the definition of the residual, which is the least squares criterion for g against $r^{(b-1)}$. The second claim follows by induction: it holds at $b = 0$,

and subtracting the two lines of (8) preserves it. □

Remark. Boosting therefore has nothing to do with the variance argument of §1. Bagging averages members fitted to the same target and lowers the second term of (1); boosting adds members fitted to what is left over, and each one lowers the training error. It is a device against bias, and the base learner is chosen small for that reason where bagging chooses it large.

3.2 Boosting a linear learner

The base learner in Definition 12.14 is a tree, whose splits depend on the response, so the map from data to fit is not linear and cannot be written as a matrix. If the learner is replaced by a fixed linear smoother the whole procedure collapses to one matrix, and everything about λ and B can be read off its spectrum.

Theorem 12.16 (The boosting operator). Let $\mathbf{S}$ be symmetric with eigenvalues $s_1, \dots, s_n$ in $[0, 1]$, and take the base learner to be $\mathbf{r} \mapsto \mathbf{S}\mathbf{r}$. Then for every $B \geq 0$

$$\mathbf{r}^{(B)} = (\mathbf{I} - \lambda \mathbf{S})^B \mathbf{y}, \quad \hat{\mathbf{f}}^{(B)} = \{\mathbf{I} - (\mathbf{I} - \lambda \mathbf{S})^B\} \mathbf{y}. \quad (9)$$

The boosting operator is symmetric with eigenvalues $1 - (1 - \lambda s_k)^B$, which for $\lambda \in (0, 1]$ increase

in B from 0 to 1 at every k with $s_k > 0$ and stay at 0 where $s_k = 0$. Hence

$$\text{df}(B) = \sum_{k=1}^n \{1 - (1 - \lambda s_k)^B\} \quad (10)$$

increases from 0 to $\text{rank}(\mathbf{S})$, and $\hat{\mathbf{f}}^{(B)}$ converges to the orthogonal projection of $\mathbf{y}$ on the range of $\mathbf{S}$.

Proof. Induction. At $B = 0$ both statements read $\mathbf{r}^{(0)} = \mathbf{y}$ and $\hat{\mathbf{f}}^{(0)} = \mathbf{0}$. By (8) the base learner returns $\mathbf{S}\mathbf{r}^{(b-1)}$, so $\mathbf{r}^{(b)} = \mathbf{r}^{(b-1)} - \lambda \mathbf{S}\mathbf{r}^{(b-1)} = (\mathbf{I} - \lambda \mathbf{S})\mathbf{r}^{(b-1)}$, which gives the first identity; and $\hat{\mathbf{f}}^{(b)} = \mathbf{y} - \mathbf{r}^{(b)}$ throughout by Proposition 12.15, which gives the second.

By the spectral theorem (prerequisites) write $\mathbf{S} = \mathbf{U} \text{diag}(s_k) \mathbf{U}'$ with $\mathbf{U}$ orthogonal. Then $(\mathbf{I} - \lambda \mathbf{S})^B = \mathbf{U} \text{diag}\{(1 - \lambda s_k)^B\} \mathbf{U}'$ and the boosting operator is $\mathbf{U} \text{diag}\{1 - (1 - \lambda s_k)^B\} \mathbf{U}'$, symmetric with the stated eigenvalues. For $\lambda \in (0, 1]$ and $s_k \in [0, 1]$ one has $1 - \lambda s_k \in [0, 1)$ whenever $s_k > 0$, so $(1 - \lambda s_k)^B$ decreases to 0 and the eigenvalue increases to 1; where $s_k = 0$ it is 0 for every B . Summing the eigenvalues gives (10), whose limit is the number of non-zero s_k , and the

operator converges to $\mathbf{U} \text{diag}(\mathbf{1}\{s_k > 0\})\mathbf{U}'$, the orthogonal projection on the range of $\mathbf{S}$. $\square$

Remark. Read as a statement about tuning: boosting starts at the null fit and moves towards the base learner's saturated fit, and B is what says how far. Nothing stops it early, so unlike bagging the number of rounds is a genuine tuning parameter and too many overfit. That is (10): the effective degrees of freedom grow with B without ever being asked to stop.

Corollary 12.17 (λ and B trade off). Fix an eigenvalue $s > 0$ and a target level $c \in (0, 1)$. The number of rounds needed for $1 - (1 - \lambda s)^B = c$ is

$$B = \frac{\ln(1 - c)}{\ln(1 - \lambda s)} \sim \frac{-\ln(1 - c)}{\lambda s} \quad \text{as } \lambda \downarrow 0, \quad (11)$$

so λB tends to a constant: halving the shrinkage roughly doubles the rounds required, and the two parameters are close to a single one.

Proof. Solve $(1 - \lambda s)^B = 1 - c$ by taking logarithms. As $\lambda \downarrow 0$, $\ln(1 - \lambda s) = -\lambda s + O(\lambda^2)$ by Taylor's theorem (prerequisites). $\square$

3.3 Depth and interaction

Proposition 12.18 (A tree with d splits has interaction order at most d). Let T have d splits. Then $\hat{f}_T$ is a sum of terms each depending on at most d coordinates of $\mathbf{x}$. In particular a stump, $d = 1$, gives a function of one coordinate, and a boosted ensemble of stumps is additive: $\hat{f}^{(B)}(\mathbf{x}) = \sum_{j=1}^p g_j(x_j)$.

Proof. A leaf of T is reached by following at most d splits, so its box is an intersection of at most d half-lines and its indicator is a function of the at most d coordinates those splits used. By Unit 11 the tree is a sum of leaf constants times such indicators, which is the first claim. For $d = 1$ each of the two leaves involves one coordinate, the same one, so a stump is a function of a single x_j ; a sum of such functions collects into $\sum_j g_j(x_j)$, and (8) produces exactly such a sum. $\square$

Corollary 12.19 (An exact test for additivity). If $\hat{f}(\mathbf{x}) = \sum_j g_j(x_j)$ then for any $a \neq a'$ and $b \neq b'$ in coordinates 1 and 2, the second difference

$$\hat{f}(a, b, \cdot) - \hat{f}(a, b', \cdot) - \hat{f}(a', b, \cdot) + \hat{f}(a', b', \cdot) \quad (12)$$

is exactly 0, the other coordinates being held fixed.

Proof. Every term g_j with $j \geq 3$ is unchanged by all four evaluations and cancels in pairs; $g_1(a)$ appears once with each sign, as does $g_1(a')$, $g_2(b)$ and $g_2(b')$. $\square$

Remark. So d is not a smoothness knob but a statement about which interactions the model is allowed to represent, and Corollary 12.19 turns it into something checkable: §4.6 computes (12) for depth one and depth two and gets 7×10^{-15} and 0.79.

4 Computation: Formulas Against Package Output

ISLR §8.3.3–8.3.4.

Boston from ISLR2 again, and a simulated design in which the truth is known: $p = 10$ predictors, one with coefficient 4, four with coefficient 1, five with coefficient 0. Simulation is what makes σ^2 and ρ of Theorem 12.1 measurable, since both are variances over the draw of the training set and a single data set supplies only one draw.

```
1 suppressMessages({library(ISLR2); library(randomForest); library(gbm)})
2 p <- 10; ntr <- 200; beta <- c(4, rep(1, 4), rep(0, p - 5))
```

```

3 gen <- function(m) { X <- data.frame(matrix(rnorm(m*p), m, p))
4   f <- as.numeric(as.matrix(X) %*% beta)
5   list(X = X, y = f + rnorm(m, 0, 2), f = f) }
6 c(p = p, n_train = ntr, strong = beta[1], moderate = beta[2],
7   noise_predictors = sum(beta == 0))

```

```

##           p          n_train          strong
##          10          200           4
## moderate noise_predictors
##           1           5

```

4.1 The variance identity (§1.1)

Corollary 12.2 is exact, so it should hold to machine precision. The matrix $\mathbf{Q}$ has one row per simulated training set and one column per tree of a forest fitted to it, evaluated at a fixed test point.

```

1 set.seed(1); B <- 60; M <- 150; nte <- 25
2 te <- gen(nte)
3 fitall <- function(mt) {
4   A <- array(0, c(M, B, nte))
5   for (k in 1:M) { d <- gen(ntr)
6     rf <- randomForest(x = d$X, y = d$y, mtry = mt, ntree = B)

```

```

7   A[k, , ] <- t(predict(rf, te$X, predict.all = TRUE)$individual) }
8   A }
9   A5 <- fitall(5)
10  per <- sapply(1:n(te), function(j) { Q <- A5[, , j]; S <- cov(Q)
11    s2 <- mean(diag(S)); cb <- mean(S[upper.tri(S)])
12    c(s2/B + (B - 1)/B*cb, var(rowMeans(Q))) })
13  c(identity_max_gap = max(abs(per[1, ] - per[2, ])),
14    average_ensemble_variance = mean(per[2, ]))

```

```

##           identity_max_gap average_ensemble_variance
##           2.220e-16           8.191e-01

```

4.2 Out-of-bag error (§1.3)

Theorem 12.7(i) on `Boston`, and the out-of-bag error against a held-out test error on one split.

```

1  set.seed(1); n <- nrow(Boston); tr <- sample(n, n/2)
2  rf <- randomForest(medv ~ ., Boston[tr, ], mtry = 4, ntree = 500,
3    xtest = Boston[-tr, -13], ytest = Boston$medv[-tr])
4  c(oob_fraction = (1 - 1/length(tr))^length(tr), limit = exp(-1),
5    oob_mse = tail(rf$mse, 1), test_mse = tail(rf$test$mse, 1))

```

##	oob_fraction	limit	oob_mse	test_mse
##	0.3672	0.3679	10.1709	18.5167

The two differ by a factor of 1.8 on this split, which is not evidence of anything: a test error computed on 253 observations is itself variable. Averaging over splits is what the comparison requires.

```

1 set.seed(5)
2 R <- t(replicate(25, { i <- sample(n, n/2)
3   g <- randomForest(medv ~ ., Boston[i, ], mtry = 4, ntree = 300,
4     xtest = Boston[-i, -13], ytest = Boston$medv[-i])
5   c(tail(g$mse, 1), tail(g$test$mse, 1)) }))
6 c(mean_oob = mean(R[, 1]), mean_test = mean(R[, 2]),
7   ratio = mean(R[, 1])/mean(R[, 2]), sd_of_test_mse = sd(R[, 2]))

```

##	mean_oob	mean_test	ratio	sd_of_test_mse
##	13.865	12.385	1.120	2.732

Averaged, the out-of-bag error sits 12% above the test error, the direction Theorem 12.7(iii) predicts, and the standard deviation of the test error across splits is large enough to explain the single-split discrepancy on its own.

4.3 Decorrelation (§2.1)

The two quantities of Theorem 12.1 measured separately as m varies, on the simulated design where independent training sets are available.

```

1 summ <- function(mt) { A <- fitall(mt)
2   q <- sapply(1:n(te), function(j) { Q <- A[, , j]; S <- cov(Q); en <- rowMeans(Q)
3     c(mean(diag(S)), mean(S[upper.tri(S)])/mean(diag(S)),
4       var(en), (mean(en) - te$f[j])^2) })
5   c(mtry = mt, sigma2 = mean(q[1, ]), rho = mean(q[2, ]),
6     variance = mean(q[3, ]), bias_squared = mean(q[4, ]),
7     total = mean(q[3, ]) + mean(q[4, ])) }
8 set.seed(1); t(sapply(c(1, 2, 3, 5, 7, 10), summ))

```

##	mtry	sigma2	rho	variance	bias_squared	total
## [1,]	1	17.544	0.02898	0.7883	7.143	7.932
## [2,]	2	15.193	0.03854	0.8211	3.875	4.696
## [3,]	3	12.755	0.04673	0.7948	2.490	3.284
## [4,]	5	9.991	0.06867	0.8356	1.425	2.261
## [5,]	7	8.721	0.08671	0.8901	1.114	2.004
## [6,]	10	7.839	0.12707	1.1218	1.040	2.162

Single trees improve as m grows, σ^2 falling by more than half from $m = 1$ to $m = p$; they also become more alike, ρ rising fourfold. The ensemble variance follows ρ and is smallest at $m = 1$, so on variance alone the most restricted forest wins. It does not win on error: the squared bias falls by a factor of seven over the same range, because trees denied the strong predictor fit the truth worse, and the sum of the two is smallest at $m = 7$. That is Unit 1's decomposition applied to the tuning parameter, and neither term alone locates the optimum.

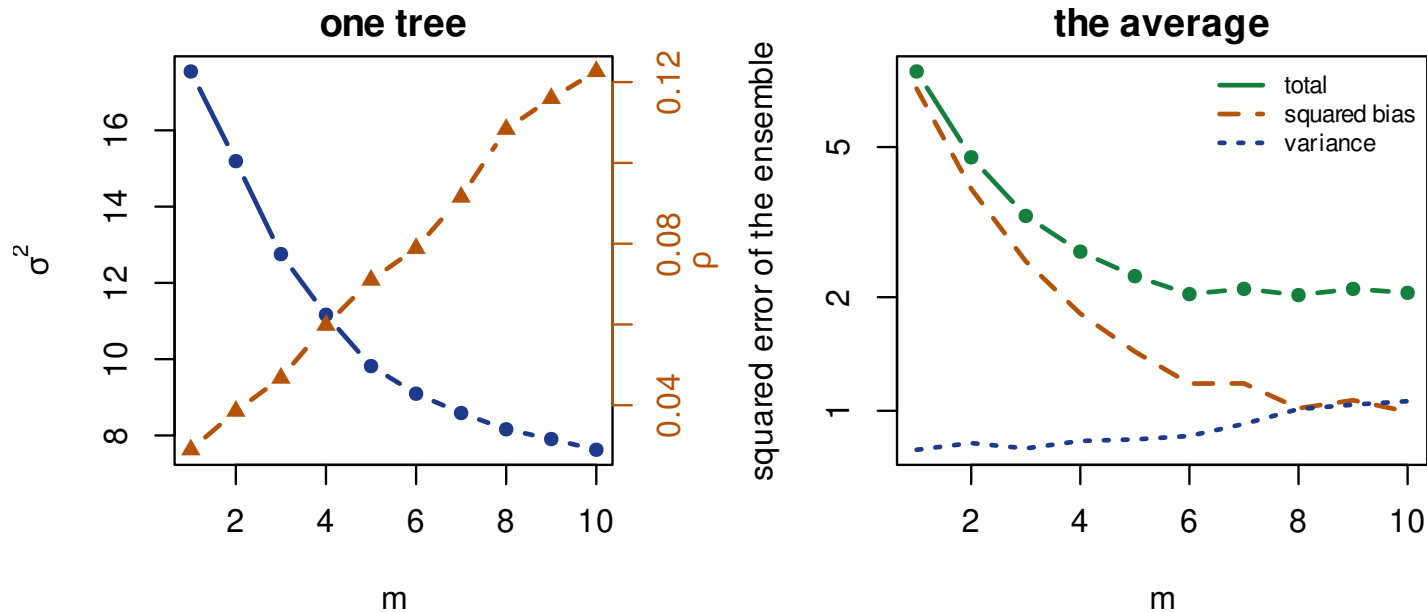


Figure 12.1: Left: as more predictors are made available at each split the individual trees get better (solid, left axis) and more alike (dashed, right axis), which is the whole of Definition 12.12. Right, on a logarithmic scale: the ensemble's variance rises with m and its squared bias falls, so the total turns. Neither term alone locates the best m .

4.4 Variable importance (§1.4)

Proposition 12.9: the impurity gains of one tree add to what the tree explains.

```

1 suppressMessages(library(tree))
2 t1 <- tree(medv ~ ., Boston); fr <- t1$frame
3 id <- as.integer(rownames(fr)); lf <- fr$var == "<leaf>"
4 gain <- sapply(id[!lf], function(t) fr$dev[match(t, id)] -
5   fr$dev[match(2*t, id)] - fr$dev[match(2*t + 1, id)])
6 c(sum_of_gains = sum(gain),
7   root_minus_tree = fr$dev[1] - sum(fr$dev[lf]),
8   gap = sum(gain) - (fr$dev[1] - sum(fr$dev[lf])))

```

```

##      sum_of_gains root_minus_tree      gap
##           35983           35983           0

```

```

1 round(sort(tapply(gain, droplevels(fr$var[!lf]), sum), decreasing = TRUE), 1)

```

```

##      rm    lstat    dis    crim ptratio
## 23842.4  8037.5  2520.3  1136.8   445.5

```

```

1 set.seed(3)
2 rf2 <- randomForest(medv ~ ., Boston, mtry = 4, ntree = 300, importance = TRUE)

```

```
3 im <- importance(rf2)
4 round(head(im[order(-im[, 2]), ], 5), 1)
```

##	%IncMSE	IncNodePurity
## lstat	24.1	12787
## rm	32.3	12783
## nox	14.2	2899
## ptratio	12.5	2574
## dis	14.6	2562

4.5 Bagging, forests and boosting compared (§2.2)

Two truths in six coordinates: one linear, one the exclusive-or of two boxes that Unit 11 showed greedy splitting cannot see at the first split. The comparison is against the noise free truth, so what is measured is approximation error and not the irreducible part.

```
1 sim <- function(kind, m) { X <- data.frame(matrix(runif(m*6), m, 6))
2   f <- if (kind == "linear") 3*X[, 1] + 2*X[, 2] - X[, 3]
3     else 4*((X[, 1] > .5) == (X[, 2] > .5))
4   data.frame(X, y = f + rnorm(m, 0, .5), truth = f) }
5 set.seed(4)
6 t(sapply(c("linear", "boxes"), function(kind) {
```

```

7  a <- sim(kind, 400); e <- sim(kind, 2000)
8  mse <- function(v) mean((v - e$truth)^2)
9  c(bagging = mse(predict(randomForest(y ~ . - truth, a, mtry = 6,
10                                     ntree = 400), e)),
11    forest_m2 = mse(predict(randomForest(y ~ . - truth, a, mtry = 2,
12                                     ntree = 400), e)),
13    boosting = mse(predict(gbm(y ~ . - truth, data = a,
14                               distribution = "gaussian", n.trees = 2000, shrinkage = 0.02,
15                               interaction.depth = 2, bag.fraction = 1, verbose = FALSE),
16                               e, n.trees = 2000)),
17    linear = mse(predict(lm(y ~ . - truth, a), e))) })))

```

```

##          bagging forest_m2 boosting   linear
## linear 0.06746   0.06864   0.05594 0.005949
## boxes  0.36385   1.27150   0.58057 4.076029

```

On the linear truth the linear model is an order of magnitude better than any ensemble, which is Unit 11's staircase theorem: no amount of averaging repairs an approximation error. On the box truth the ordering reverses completely, and the forest with $m = 2$ is more than three times worse than plain bagging, because hiding coordinates makes the pairing of a split on x_1 with a split on x_2 rarer.

4.6 Boosting (§3)

Theorem 12.16 with $\mathbf{S}$ the projection on a fixed six-cell partition, which is a legitimate linear smoother by Unit 11.

```

1 set.seed(3); m <- 120; xb <- runif(m); yb <- sin(6*xb) + rnorm(m, 0, .3)
2 grp <- cut(xb, quantile(xb, seq(0, 1, length.out = 7)), include.lowest = TRUE)
3 D <- model.matrix(~ grp - 1); S <- D %*% solve(crossprod(D)) %*% t(D)
4 mpow <- function(A, k) { R <- diag(nrow(A)); for (i in seq_len(k)) R <- R %*% A; R }
5 run <- function(lam, Bn) { f <- rep(0, m); r <- yb
6   for (b in 1:Bn) { g <- S %*% r; f <- f + lam*g; r <- r - lam*g }; f }
7 ev <- eigen(S, symmetric = TRUE)$values
8 t(sapply(list(c(.1, 5), c(.3, 20), c(.05, 100)), function(z) {
9   f1 <- run(z[1], z[2])
10  f2 <- (diag(m) - mpow(diag(m) - z[1]*S, z[2])) %*% yb
11  c(lambda = z[1], B = z[2], max_gap = max(abs(f1 - f2)),
12    edf = sum(1 - (1 - z[1]*ev)^z[2])) })))

```

##	lambda	B	max_gap	edf
## [1,]	0.10	5	4.441e-16	2.457
## [2,]	0.30	20	1.332e-15	5.995
## [3,]	0.05	100	3.664e-15	5.964

```
1 c(rank_of_S = sum(ev > 0.5), distinct_eigenvalues = length(unique(round(ev, 8))))
```

```
##           rank_of_S distinct_eigenvalues
##                6                2
```

The identity holds to machine precision and the effective degrees of freedom (10) climb towards the rank of $\mathbf{S}$: boosting is walking from the null fit to the saturated fit of its base learner, and B says how far. Corollary 12.17 then says λ and B are nearly one parameter.

```
1 set.seed(7)
2 gs <- function(m) { X <- data.frame(matrix(runif(m*6), m, 6))
3   data.frame(X, y = 2*X[, 1] + sin(6*X[, 2]) + 3*X[, 1]*X[, 2] +
4     rnorm(m, 0, .4)) }
5 gtr <- gs(500); gte <- gs(2000); Bs <- seq(50, 4000, by = 50)
6 t(sapply(c(0.01, 0.02, 0.05), function(lam) {
7   g <- gbm(y ~ ., data = gtr, distribution = "gaussian",
8     n.trees = 4000, shrinkage = lam, interaction.depth = 2,
9     bag.fraction = 1, verbose = FALSE)
10   mse <- colMeans((predict(g, gte, n.trees = Bs) - gte$y)^2)
11   c(lambda = lam, best_B = Bs[which.min(mse)],
```

```
12 lambda_times_B = lam*Bs[which.min(mse)], min_mse = min(mse)) }))
```

```
##      lambda best_B lambda_times_B min_mse
## [1,]    0.01    1700             17 0.1927
## [2,]    0.02     800             16 0.1926
## [3,]    0.05     300             15 0.1924
```

The products are 17, 16 and 15 and the three minima agree to three decimal places: a fivefold change in the shrinkage buys nothing except a fivefold change in the running time.

Corollary 12.19 is an exact test, so a depth-one ensemble must return zero.

```
1 fitg <- function(dep) gbm(y ~ ., data = gtr, distribution = "gaussian",
2   n.trees = 800, shrinkage = 0.05, interaction.depth = dep,
3   bag.fraction = 1, verbose = FALSE)
4 mixed <- function(g) { nd <- gte[rep(1, 4), ]
5   nd$X1 <- c(.2, .2, .8, .8); nd$X2 <- c(.2, .8, .2, .8)
6   f <- predict(g, nd, n.trees = 800); f[1] - f[2] - f[3] + f[4] }
7 signif(c(depth_1 = mixed(fitg(1)), depth_2 = mixed(fitg(2))), 3)
```

```
## depth_1 depth_2
## 6.66e-15 7.91e-01
```

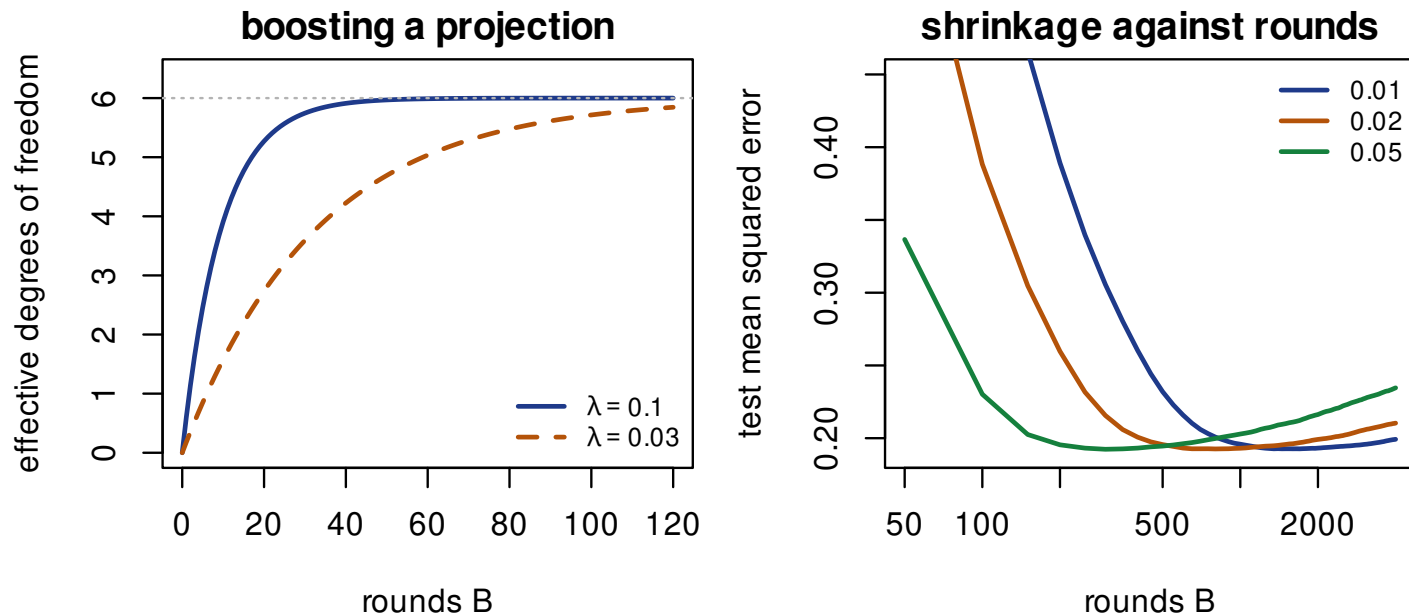


Figure 12.2: Left: the effective degrees of freedom (10) of boosting a rank-six projection, rising to the rank; a third of the shrinkage needs about three times the rounds to reach the same place, which is Corollary 12.17. Right: test error against rounds for three shrinkages; the three curves are the same curve shifted along a logarithmic axis, and their minima occur at nearly equal λB .

5 Pitfalls

5.1 Treating the number of trees as a tuning parameter

By Theorem 12.1 the ensemble variance decreases in B and nothing in bagging increases with it, so B is a computing budget. Boosting is the opposite case, and the two are often confused.

```

1 set.seed(8); a <- sim("boxes", 400); e <- sim("boxes", 2000)
2 Bv <- c(5, 25, 100, 400, 1600)
3 bg <- rowMeans(replicate(10, sapply(Bv, function(b)
4   mean((predict(randomForest(y ~ . - truth, a, mtry = 6, ntree = b),
5     e) - e$truth)^2))))
6 gz <- gbm(y ~ . - truth, data = a, distribution = "gaussian",
7   n.trees = 8000, shrinkage = 0.1, interaction.depth = 3,
8   bag.fraction = 1, verbose = FALSE)
9 rbind(trees = Bv, bagging = bg, rounds = 5*Bv,
10   boosting = sapply(5*Bv, function(b)
11     mean((predict(gz, e, n.trees = b) - e$truth)^2)))

```

##	[,1]	[,2]	[,3]	[,4]	[,5]
## trees	5.000	25.000	100.0000	400.0000	1600.0000
## bagging	1.296	1.095	0.9995	0.9785	0.9757

```
## rounds    25.000 125.000 500.0000 2000.0000 8000.0000
## boosting   3.188   0.914   0.6273   0.6595   0.6759
```

The bagging row falls and flattens; the boosting row falls, reaches its minimum at 500 rounds, and then climbs. Averaging over ten forests is what makes the first row monotone: a single forest at $B = 5$ is too variable to read.

Diagnostic. A cross-validation loop over the number of trees in a random forest, or a boosted model reported at whatever B the software defaulted to. **Fix.** For bagging and forests, raise B until the out-of-bag error stops moving and stop thinking about it. For boosting, B must be chosen, because Theorem 12.16 says the fit keeps moving towards the saturated fit of the base learner and (10) keeps growing.

5.2 Reading impurity importance as evidence

Proposition 12.11 gives permutation importance a zero for a useless predictor. Nothing of the sort holds for the impurity measure of Definition 12.8. Below, the response is independent of all four predictors.

```
1 set.seed(2); mm <- 500
2 dn <- data.frame(
```

```
3 continuous = runif(mm),
4 twenty_levels = factor(sample(1:20, mm, TRUE)),
5 four_levels = factor(sample(1:4, mm, TRUE)),
6 binary = factor(sample(0:1, mm, TRUE)),
7 y = rnorm(mm))
8 rfn <- randomForest(y ~ ., dn, ntree = 500, importance = TRUE)
9 round(importance(rfn), 2)
```

##	%IncMSE	IncNodePurity
## continuous	2.83	49.39
## twenty_levels	4.09	64.98
## four_levels	3.75	15.99
## binary	2.71	5.75

Diagnostic. An importance ranking that reproduces the ranking of the predictors by number of distinct values. **Fix.** Report permutation importance, and read impurity importance as a description of how the trees were built rather than of the population. Here the twenty-level factor scores eleven times the binary one on impurity while the response is independent of both; the permutation column separates them by less than a factor of two.

5.3 Comparing out-of-bag with test error on one split

§4.2 found a factor of 1.8 on one split of **Boston** and 1.12 averaged over twenty-five. The first number is noise.

Diagnostic. A claim that the out-of-bag error is optimistic or pessimistic, based on one split. **Fix.** The standard deviation of the test error across splits is printed in §4.2 and is large enough to account for the gap. Theorem 12.7 says the direction of the bias is upward and its size is of order $1/B$, which at $B = 300$ is far below the split-to-split variability, so a single comparison cannot detect it.

5.4 Bagging a low-variance learner

```
1 set.seed(9)
2 R1 <- t(replicate(20, { a1 <- sim("linear", 400); e1 <- sim("linear", 2000)
3   P <- replicate(60, { i <- sample(nrow(a1), replace = TRUE)
4     predict(lm(y ~ . - truth, a1[i, ]), e1) })
5   c(mean((predict(lm(y ~ . - truth, a1), e1) - e1$truth)^2),
6     mean((rowMeans(P) - e1$truth)^2),
7     mean((predict(randomForest(y ~ . - truth, a1, mtry = 6, ntree = 60),
8       e1) - e1$truth)^2)) }))
9 c(single_lm = mean(R1[, 1]), bagged_lm = mean(R1[, 2]),
```

```
10 bagged_tree = mean(Rl[, 3]), bagging_gain = mean(Rl[, 1])/mean(Rl[, 2]))
```

##	single_lm	bagged_lm	bagged_tree	bagging_gain
##	0.004851	0.004858	0.063644	0.998532

Diagnostic. Bagging applied to a method that was not unstable to begin with. **Fix.** By Proposition 12.4 bagging removes $(1 - \rho)\sigma^2(1 - 1/B)$ and no more. A least squares fit on 400 observations with six predictors has small σ^2 and its bootstrap replicates are nearly identical, so ρ is close to one and there is almost nothing to remove. Averaged over twenty training sets the bagged linear model is not better than the single one at all, while both beat the bagged trees by a factor of thirteen. Bagging is a variance device and needs a variance to work on.

5.5 Boosting without holding anything out

```
1 set.seed(10); ab <- sim("boxes", 300); eb <- sim("boxes", 2000)
2 gb <- gbm(y ~ . - truth, data = ab, distribution = "gaussian",
3           n.trees = 8000, shrinkage = 0.1, interaction.depth = 4,
4           bag.fraction = 1, verbose = FALSE)
5 Bx <- c(50, 200, 800, 3200, 8000)
6 rbind(rounds = Bx,
7       training_mse = sapply(Bx, function(b)
```

```

8      mean((predict(gb, ab, n.trees = b) - ab$y)^2)),
9      test_mse = sapply(Bx, function(b)
10     mean((predict(gb, eb, n.trees = b) - eb$y)^2)))

```

```

##           [,1]      [,2]      [,3]      [,4]      [,5]
## rounds    50.000 200.0000 800.000000 3.200e+03 8.000e+03
## training_mse 0.689   0.1402   0.008478 2.731e-06 1.432e-12
## test_mse     1.411   0.8929   0.818326 8.260e-01 8.260e-01

```

Diagnostic. A training error that keeps falling while the model is declared to be improving. **Fix.** The training error of a boosted model decreases in B by construction, since each round fits the current residuals, so it carries no information about when to stop. Choose B by cross-validation or a held-out set. The test column turns while the training column does not.

6 Exercises

Exercise 12.1. An ensemble member has prediction variance $\sigma^2 = 9$ at a given point and the members have pairwise correlation $\rho = 0.4$. Find the variance of an average of 10, of 100 and of infinitely many. What fraction of a single member's variance can averaging remove?

Solution. By Theorem 12.1, $\text{var} = \rho\sigma^2 + (1 - \rho)\sigma^2/B = 3.6 + 5.4/B$: at $B = 10$ it is 4.14, at $B = 100$ it is 3.654, and the limit is 3.6.

Averaging can remove at most $(1 - \rho)\sigma^2 = 5.4$, which is 60% of the single member's 9; the remaining 40% is the floor $\rho\sigma^2$. Note also that $B = 10$ already achieves 4.14 against a limit of 3.6, so nine tenths of the available reduction is bought by the first ten members. That is why B is a computing budget and not a tuning parameter. ■

Exercise 12.2. Why does a random forest with $m < p$ often beat bagging, and why is it not guaranteed to?

Solution. By Theorem 12.1 the limiting variance is $\rho\sigma^2$. Restricting each split to m of the p predictors makes the trees less alike, so ρ falls; §4.3 measures ρ rising from 0.029 at $m = 1$ to 0.127 at $m = p$ on the simulated design.

It is not guaranteed because the same restriction removes candidates from every split and so raises σ^2 , which the same table shows falling from 17.5 to 7.8 as m grows, and because variance is not the whole error. On that design the ensemble variance is smallest at $m = 1$ and the squared bias smallest at $m = p$, and the sum is smallest at $m = 7$: the optimum is a property of the data

and no argument here locates it. §4.5 exhibits an outright failure: on an exclusive-or truth in two coordinates the structure needs a split on x_1 followed by one on x_2 , and hiding coordinates makes that pairing rarer, so $m = 2$ is three times worse than bagging. ■

Exercise 12.3. With $n = 1000$ and $B = 500$, how many trees on average is an out-of-bag prediction an average of? Is the out-of-bag error optimistic or pessimistic for the full ensemble, and by roughly how much?

Solution. By Theorem 12.7(i) the expected number is $B(1 - 1/n)^n = 500(0.9995)^{1000}$, which is $500(0.3677) = 183.8$, close to $B/e = 183.9$.

It is pessimistic. The out-of-bag prediction has the same bias as the full ensemble and larger variance, by $(1 - \rho)\sigma^2(1/184 - 1/500) = (1 - \rho)\sigma^2(0.00343)$ from part (iii). Relative to the ensemble's own variance $\rho\sigma^2 + (1 - \rho)\sigma^2/500$ that is a small correction, and it vanishes like $1/B$. §4.2 measures 12% on **Boston** at $B = 300$, which also contains whatever the data contribute beyond the formula. ■

Exercise 12.4. A random forest reports impurity importances 40, 25, 20, 15 for four predictors, summing to 100. What do these numbers mean, and what would you check before reporting them as evidence that the first predictor matters most?

Solution. By Proposition 12.9 the impurity gains of a tree sum to $R(\text{root}) - R(T)$, the total cost the tree removes, so the four numbers are shares of the fit the ensemble achieved, averaged over trees. They describe how the trees were built.

What to check is the number of distinct values each predictor takes. A predictor with many cut-points reduces the training cost by more than one with few, whatever its relation to the response, and §5.2 shows four predictors independent of the response scoring 65, 49, 16, 6 purely by cardinality. Permutation importance has no such defect: by Proposition 12.11 a predictor independent of (Y, X_{-j}) has population importance exactly zero. Report that, or at least both. ■

Exercise 12.5. Twenty-one classifiers each get a given observation right with probability 0.6. Bound the probability the majority vote is right if they are independent. Why is the bound not available for a bagged ensemble?

Solution. By Proposition 12.5 the failure probability is at most $\pi(1 - \pi)/\{B(\pi - \frac{1}{2})^2\} = 0.24/\{21(0.01)\} = 1.14$, which is vacuous: Chebyshev gives nothing at these numbers, and $B > 24$ is needed before the bound says anything. The exact binomial probability of at least eleven successes is 0.826, so the vote does help; the bound is simply weak.

For a bagged ensemble the hypothesis fails outright. The members are fitted to resamples of one data set and are positively correlated, which is what $\rho > 0$ in Theorem 12.1 records. A correlated vote behaves like a smaller independent vote, and in the extreme $\rho = 1$ all members agree and the ensemble is one classifier. ■

Exercise 12.6. Boosting is run with a base learner that projects onto a fixed subspace of dimension r , with shrinkage λ . What is the fit after B rounds, what does it converge to, and what are its effective degrees of freedom?

Solution. By Theorem 12.16 the fit is $\{\mathbf{I} - (\mathbf{I} - \lambda\mathbf{S})^B\}\mathbf{y}$. A projection has eigenvalues 1 with multiplicity r and 0 otherwise, so the boosting operator has eigenvalues $1 - (1 - \lambda)^B$ on the subspace and 0 off it. The fit is therefore $\{1 - (1 - \lambda)^B\}\mathbf{S}\mathbf{y}$, a shrunk version of the projection, converging to $\mathbf{S}\mathbf{y}$ itself.

The effective degrees of freedom are $r\{1 - (1 - \lambda)^B\}$, rising from 0 to r . So boosting a fixed linear learner adds nothing the learner could not already do; it only controls how far along the way the fit is stopped. What makes boosting trees different is that the base learner is refitted to the current residuals each round, so the subspace itself changes. ■

Exercise 12.7. A boosted model with $\lambda = 0.05$ is best at $B = 300$ rounds. Roughly what B would be best at $\lambda = 0.005$, and why does the answer matter in practice?

Solution. By Corollary 12.17 the product λB is nearly constant, so ten times smaller shrinkage needs about ten times the rounds: $B \approx 3000$. §4.6 measures the products 17, 16 and 15 at $\lambda = 0.01, 0.02$ and 0.05 , with essentially the same minimum error at all three.

It matters because the two parameters are close to one parameter, so searching both on a grid mostly wastes computation, and because the cost of the fit is linear in B : the small shrinkage that is often recommended buys nothing here except a tenfold longer run. The right reading of Corollary 12.17 is that λ should be as large as the error curve tolerates. ■

Exercise 12.8. A boosted ensemble is fitted with `interaction.depth = 1`. What class of functions can it represent, and how would you confirm on a fitted object that it is in that class?

Solution. By Proposition 12.18 a stump is a function of a single coordinate, so a sum of stumps is additive: $\hat{f}(\mathbf{x}) = \sum_j g_j(x_j)$, with no interaction terms of any order.

Corollary 12.19 makes it checkable exactly. Evaluate the fitted function at the four corners of a rectangle in coordinates 1 and 2, holding the rest fixed, and form $\hat{f}(a, b) - \hat{f}(a, b') - \hat{f}(a', b) + \hat{f}(a', b')$. For an additive function every term cancels and the answer is exactly zero. §4.6 gets 7×10^{-15} at depth one and 0.79 at depth two. ■

Exercise 12.9. A response is exactly linear in its predictors. Rank a linear model, bagging, a random forest and boosting, and say what changes if the truth is a partition into boxes instead.

Solution. On the linear truth the linear model wins by a wide margin: §4.5 measures 0.006 against 0.056 to 0.069 for the three ensembles, a factor of ten. The reason is Unit 11's staircase theorem, that a piecewise-constant fit approximating a linear function with J boxes carries an error of order $J^{-2/p}$. That error is bias, and Proposition 12.4 says averaging does not touch bias.

On the box truth the ordering reverses: the linear model is worst at 4.08, bagging best at 0.36, boosting 0.58, and the forest with $m = 2$ at 1.27 suffers for the reason of Exercise 12.2. The general lesson is that ensembles change the variance term and the base learner decides the bias term, so the choice between trees and a linear model is made before any ensembling is considered.



Exercise 12.10. In R: (a) verify the variance identity of Corollary 12.2 on a forest's individual tree predictions; (b) measure ρ and σ^2 as functions of `mtry`; (c) verify that boosting a fixed projection gives $\{\mathbf{I} - (\mathbf{I} - \lambda\mathbf{S})^B\}\mathbf{y}$.

Solution. (a) is §4.1, where the gap is at machine precision because Corollary 12.2 is an algebraic identity about sample moments; (b) is §4.3; (c) is §4.6, agreeing to 4×10^{-15} .

The point to watch in (b) is that ρ and σ^2 are variances over the draw of the **training set**, not over the trees within one forest. Conditionally on the data the members of a forest are independent, so estimating ρ requires many training sets, which is why that section works on simulated data rather than on **Boston**.

